Memoria del Trabajo Fin de Máster

Diseño e implementación de una plataforma conversacional corporativa basada en IA con integración documental, enrutado inteligente y automatización de incidencias

Autor: Miquel Raga

Fecha: 19/03/2026

Índice

Resumen

1. Introducción

2. Contexto y problemática

3. Objetivos del proyecto
3.1 Objetivo general
3.2 Objetivos específicos

4. Alcance del proyecto

5. Marco teórico y tecnológico

5.1 Asistentes conversacionales basados en LLM
5.2 RAG como mecanismo de contextualización
5.3 Model Context Protocol (MCP)
5.4 Plataformas tipo LibreChat
5.5 Computación en la nube y escalabilidad

6. Requisitos del sistema

6.1 Requisitos funcionales
6.2 Requisitos no funcionales

7. Arquitectura de la solución

7.1 Visión general
7.2 Frontend y experiencia conversacional
7.3 Backend y orquestación
7.4 Persistencia con MongoDB
7.5 Integración con proveedores de IA
7.6 Sistema RAG y gestión documental
7.7 Integración con Guru mediante MCP
7.8 Integración con Jira mediante MCP
7.9 Clasificación por dominio mediante archivo Markdown

8. Implementación del proyecto

8.1 Despliegue en Google Cloud
8.2 Uso de LibreChat como base tecnológica
8.3 Configuración mediante YAML
8.4 Gestión de memoria y prompts
8.5 Integración con Guru
8.6 Integración con Jira
8.7 Lógica de derivación a verticales

9. Casos de uso principales

9.1 Consulta de conocimiento corporativo
9.2 Creación automática de un ticket
9.3 Derivación directa a una vertical
9.4 Consulta apoyada en documentos adjuntos

10. Resultados y aportación del proyecto

10.1 Reducción del tiempo de desarrollo
10.2 Unificación de acceso a modelos y herramientas
10.3 Mejora del acceso al conocimiento interno
10.4 Automatización de procesos de soporte
10.5 Optimización del triage de incidencias
10.6 Modularidad y capacidad de evolución

11. Limitaciones del sistema

11.1 Dependencia de la calidad de la configuración
11.2 Clasificación basada en palabras clave
11.3 Dependencia de servicios externos
11.4 Riesgo de respuestas incorrectas
11.5 Seguridad y gestión de secretos

12. Conclusiones

13. Bibliografía orientativa, fuentes y adjuntos

13.1 Bibliografía
13.2 Adjuntos

Resumen

El presente Trabajo Fin de Máster aborda el diseño, desarrollo e implementación de una plataforma conversacional corporativa basada en inteligencia artificial, desplegada en la nube y orientada a mejorar la gestión del conocimiento interno y la automatización de procesos de soporte. El sistema se ha construido utilizando una base tecnológica ya existente, concretamente una solución tipo *LibreChat*, que proporciona una interfaz conversacional funcional y un backend capaz de gestionar conversaciones, proveedores de modelos y persistencia de datos. Sobre esta base, se ha llevado a cabo una adaptación y extensión orientada al contexto empresarial, incorporando nuevas capacidades de integración, clasificación y ejecución de acciones.

La solución desarrollada se apoya en una infraestructura desplegada en Google Cloud, utilizando MongoDB como sistema de almacenamiento principal, y permitiendo la conexión con distintos proveedores de modelos de lenguaje, como Azure y Vertex AI, así como con otros recursos externos. Esta arquitectura habilita un entorno flexible, escalable y desacoplado, en el que el sistema puede consumir distintos modelos según necesidades funcionales o criterios de coste, rendimiento o disponibilidad.

Uno de los aspectos diferenciales del proyecto es la incorporación del protocolo MCP (*Model Context Protocol*) como mecanismo de conexión con sistemas corporativos externos. En concreto, se ha integrado una fuente documental a través del MCP de Guru, que permite consultar conocimiento interno de la organización, y se ha habilitado la creación automática de incidencias mediante el MCP de Jira. Gracias a ello, el asistente no solo responde preguntas, sino que también ejecuta acciones útiles dentro del ecosistema empresarial.

Además, el sistema incorpora una lógica de clasificación basada en palabras clave definidas en un documento Markdown, cuyo objetivo es identificar el dominio funcional o vertical al que pertenece una petición del usuario. Esta clasificación permite derivar automáticamente los tickets al área correspondiente, evitando su paso previo por un helpdesk genérico y agilizando el flujo de resolución.

El trabajo se centra tanto en la arquitectura técnica como en la justificación funcional del sistema, analizando las decisiones tecnológicas adoptadas, la integración entre componentes, los criterios de seguridad y configuración, y el valor aportado en términos de eficiencia, reutilización de conocimiento y automatización de tareas. Como resultado, se obtiene una plataforma modular, extensible y alineada con las necesidades reales de una organización que busca incorporar inteligencia artificial de forma práctica y operativa.

1. Introducción

La incorporación de modelos de lenguaje de gran tamaño (*Large Language Models*, LLM) en entornos empresariales ha abierto nuevas posibilidades en ámbitos como la automatización del soporte, la consulta de conocimiento interno, la asistencia al usuario y la optimización de procesos. Sin embargo, el verdadero valor de estos modelos no reside únicamente en su capacidad para generar texto, sino en su integración efectiva con los sistemas, datos y procedimientos reales de una organización.

En un entorno corporativo, un asistente basado en IA debe ir mucho más allá de responder preguntas genéricas. Debe ser capaz de entender el contexto de negocio, acceder a documentación interna, mantener configuraciones adaptadas a la organización, respetar políticas de seguridad y, en muchos casos, ejecutar acciones útiles sobre herramientas corporativas. Además, para resultar realmente operativo, debe poder desplegarse de forma controlada, ser escalable y apoyarse en una arquitectura flexible que permita incorporar nuevos modelos y nuevas integraciones sin rehacer el sistema desde cero.

En este contexto nace el proyecto desarrollado en este Trabajo Fin de Máster. El objetivo principal ha sido construir una plataforma conversacional empresarial reutilizando un framework ya existente —en este caso una solución tipo LibreChat— y enriqueciéndolo con capacidades específicas de integración, recuperación de conocimiento y automatización de procesos. La intención no ha sido diseñar un chatbot aislado, sino una plataforma funcional que actúe como punto de entrada inteligente a distintos recursos de la organización.

El sistema se ha desplegado en Google Cloud, lo que aporta una infraestructura gestionada y preparada para su escalado. Asimismo, se ha empleado MongoDB como base de datos principal, aprovechando su flexibilidad para almacenar conversaciones, configuraciones, memoria contextual y metadatos asociados a documentos o sesiones. Sobre esta base, se han conectado diferentes proveedores de modelos de IA, como Azure y Vertex AI, permitiendo que la plataforma opere con diversos motores generativos según el caso de uso.

La propuesta incorpora también un sistema RAG (*Retrieval-Augmented Generation*), orientado a mejorar la calidad de las respuestas mediante recuperación de información contextual. A ello se suma la integración con Guru como repositorio de conocimiento corporativo y con Jira como herramienta de gestión de incidencias, ambas a través de MCP. Gracias a esta arquitectura, el asistente no se limita a dialogar: también puede consultar documentación fiable y generar tickets de forma automática en función del contenido de la conversación.

Una funcionalidad especialmente relevante del proyecto es la clasificación del dominio o vertical de una solicitud. Mediante un mecanismo sencillo pero eficaz, basado en un fichero Markdown que contiene palabras clave por ámbito funcional, el sistema determina a qué área debe derivarse una incidencia. Esto permite evitar el paso por un helpdesk generalista y dirigir el ticket directamente al equipo correspondiente, mejorando tiempos de respuesta y reduciendo carga operativa.

2. Contexto y problemática

Las organizaciones manejan un volumen cada vez mayor de información distribuida entre documentos, herramientas colaborativas, bases de conocimiento, gestores de tareas y sistemas internos. Esta dispersión provoca que muchas consultas repetitivas consuman tiempo de personal especializado, especialmente en áreas de soporte, operaciones, IT o gestión funcional. Con frecuencia, el conocimiento ya existe, pero no está accesible de forma inmediata ni en el formato adecuado para el usuario final.

A esta problemática se añade la gestión de incidencias. En muchas empresas, las solicitudes de soporte o los problemas detectados siguen un flujo centralizado: el usuario contacta con un helpdesk, el helpdesk interpreta el caso y lo deriva al área o vertical correspondiente. Este modelo tiene valor cuando no existen mecanismos automáticos de clasificación o cuando los usuarios no saben cómo canalizar la petición. Sin embargo, también introduce tiempos muertos, fricción y dependencia de una capa intermedia que, en algunos casos, puede automatizarse parcial o totalmente.

Por otro lado, el auge de los modelos generativos ha hecho que muchas organizaciones quieran incorporar asistentes conversacionales. El problema es que, si estos asistentes no tienen acceso a conocimiento interno ni capacidad de integración con sistemas corporativos, su utilidad práctica es limitada. Un chatbot puramente generalista puede ser útil para tareas lingüísticas o de ayuda genérica, pero no resuelve necesidades reales de negocio como consultar procedimientos internos, contextualizar respuestas con documentación propia o crear una incidencia directamente en el sistema de gestión correspondiente.

Ante esta situación, surge la necesidad de una plataforma capaz de reunir varias propiedades al mismo tiempo:

1. **Interfaz conversacional usable**, sin necesidad de desarrollar desde cero un frontend complejo.
2. **Integración con distintos modelos de IA**, evitando dependencia rígida de un único proveedor.
3. **Persistencia de conversaciones y configuración**, con capacidad para guardar memoria, prompts y contexto.
4. **Acceso a conocimiento corporativo**, para responder con información relevante y no únicamente con conocimiento general del modelo.
5. **Capacidad de actuación**, es decir, posibilidad de ejecutar acciones útiles como crear tickets en herramientas corporativas.
6. **Clasificación y enrutado inteligente**, para dirigir solicitudes al dominio adecuado de manera automática.
7. **Despliegue cloud y escalabilidad**, de modo que la solución pueda pasar de prototipo a entorno utilizable.

El proyecto que se presenta en esta memoria responde precisamente a esa necesidad.

3. Objetivos del proyecto

3.1 Objetivo general

Diseñar, desplegar e integrar una plataforma conversacional corporativa basada en inteligencia artificial que permita consultar conocimiento empresarial, clasificar automáticamente solicitudes según su dominio funcional y crear tickets en Jira de forma automatizada, todo ello sobre una infraestructura cloud escalable y configurable.

3.2 Objetivos específicos

1. **Aprovechar una base funcional existente** mediante el uso de una solución tipo LibreChat, reduciendo el tiempo de desarrollo del frontend y parte del backend.
2. **Desplegar la plataforma en Google Cloud**, garantizando accesibilidad, estabilidad y capacidad de escalado.
3. **Configurar MongoDB como base de datos principal**, para almacenar conversaciones, configuraciones, memoria y metadatos relacionados.
4. **Integrar distintos proveedores de modelos** como Azure y Vertex AI, permitiendo flexibilidad en el consumo de servicios de IA.
5. **Conectar el sistema con fuentes de conocimiento corporativo** mediante el MCP de Guru, habilitando consultas basadas en documentación interna.
6. **Incorporar la creación automatizada de incidencias** mediante el MCP de Jira, de forma que el asistente pueda ejecutar acciones operativas.
7. **Desarrollar una lógica de clasificación por dominios** basada en palabras clave definidas en un archivo Markdown, con el fin de determinar la vertical a la que debe dirigirse una petición.
8. **Reducir dependencia del helpdesk como punto de triage inicial**, dirigiendo las incidencias directamente al área competente cuando sea posible.
9. **Establecer una arquitectura modular y extensible**, apta para incorporar nuevas integraciones, modelos o reglas en el futuro.
10. **Documentar técnica y funcionalmente la solución**, justificando decisiones de diseño, alcance, ventajas y limitaciones.

4. Alcance del proyecto

El alcance del presente trabajo se centra en la construcción de una plataforma funcional de asistencia conversacional corporativa con integración documental y automatización de tickets. El proyecto cubre los siguientes elementos:

- Despliegue de la solución en infraestructura cloud.
- Configuración de la plataforma conversacional.
- Integración con base de datos MongoDB.
- Conexión con proveedores de IA externos.
- Integración vía MCP con Guru y Jira.
- Definición de una lógica de clasificación por dominios basada en palabras clave.
- Uso del sistema como punto unificado de consulta y derivación de incidencias.
- Configuración de parámetros funcionales y de seguridad a través de archivos de configuración.

Quedan fuera del alcance, o se abordan solo parcialmente, aspectos como:

- El desarrollo completo desde cero del frontend conversacional.
- El entrenamiento de modelos fundacionales propios.
- La evaluación estadística exhaustiva con grandes volúmenes de usuarios reales, si todavía no se ha desplegado masivamente.
- La implementación de un sistema avanzado de clasificación entrenado supervisadamente sobre grandes corpus etiquetados.
- La integración con la totalidad de herramientas corporativas de la organización.

Este recorte del alcance es coherente con la filosofía del proyecto: demostrar el valor práctico de una solución aplicable y extensible, priorizando la integración real frente a la investigación puramente experimental.

5. Marco teórico y tecnológico

5.1 Asistentes conversacionales basados en LLM

Los LLM han transformado la forma en que interactuamos con sistemas software. Su capacidad para comprender lenguaje natural, generar respuestas coherentes y adaptarse a múltiples tareas los convierte en una base idónea para asistentes virtuales empresariales. No obstante, estos modelos presentan limitaciones relevantes: conocimiento desactualizado, alucinaciones, falta de acceso a fuentes internas y ausencia de capacidad de acción si no están conectados a herramientas externas.

Por ello, en entornos corporativos, los LLM suelen actuar como motor conversacional dentro de una arquitectura mayor que incorpora recuperación de información, memoria, herramientas y reglas de negocio.

5.2 RAG como mecanismo de contextualización

La generación aumentada por recuperación (*Retrieval-Augmented Generation*, RAG) es una estrategia que combina un modelo generativo con un sistema de recuperación de información. En lugar de confiar únicamente en el conocimiento paramétrico del modelo, el sistema consulta documentos o fragmentos relevantes y los introduce como contexto para generar una respuesta más precisa.

En este proyecto, el enfoque RAG es importante porque permite aprovechar documentación empresarial y documentos subidos por el usuario, mejorando la pertinencia de las respuestas y reduciendo el riesgo de invenciones.

5.3 Model Context Protocol (MCP)

El uso de MCP resulta clave para desacoplar el asistente de las herramientas corporativas concretas. MCP actúa como una capa de comunicación entre el modelo o la plataforma conversacional y servicios externos, permitiendo que el sistema consulte fuentes documentales o ejecute acciones. En el contexto de este proyecto, MCP se utiliza para:

- Obtener información desde **Guru**.
- Crear incidencias o tickets en **Jira**.

Este enfoque favorece una arquitectura más modular y mantenible, ya que cada integración puede evolucionar de forma relativamente independiente.

5.4 Plataformas tipo LibreChat

El uso de una plataforma tipo LibreChat responde a una necesidad pragmática: disponer de una interfaz conversacional robusta sin invertir tiempo excesivo en desarrollar desde cero todos los componentes de interacción. Este tipo de soluciones suele incluir:

- Interfaz web para conversaciones.
- Gestión de sesiones y usuarios.
- Configuración de proveedores de modelos.
- Persistencia de conversaciones.
- Posibilidad de adjuntar documentos.

- Extensibilidad mediante configuraciones e integraciones.

A partir de esta base, el valor del proyecto reside en la personalización y en la conexión con el ecosistema empresarial.

5.5 Computación en la nube y escalabilidad

El despliegue en Google Cloud permite abstraer gran parte de la gestión de infraestructura, facilitar el acceso remoto y sentar las bases para el escalado. En una plataforma de IA, la nube resulta especialmente útil por varios motivos:

- Elasticidad de recursos.
- Integración con servicios gestionados.
- Posibilidad de desplegar entornos reproducibles.
- Separación entre desarrollo, pruebas y producción.
- Facilidad para gestionar credenciales y seguridad.

6. Requisitos del sistema

6.1 Requisitos funcionales

El sistema debía cumplir, como mínimo, con los siguientes requisitos funcionales:

- Permitir al usuario interactuar con un asistente mediante interfaz conversacional.
- Mantener histórico de conversaciones.
- Configurar distintos proveedores de modelos de IA.
- Consultar conocimiento corporativo a través de Guru.
- Aceptar documentos y utilizarlos como contexto cuando corresponda.
- Crear tickets en Jira a partir de peticiones del usuario o conclusiones del asistente.
- Determinar el dominio funcional de una incidencia mediante una clasificación basada en palabras clave.
- Enrutar el ticket directamente a la vertical correspondiente.
- Gestionar prompts, memoria y configuraciones del sistema.
- Respetar un modelo de despliegue seguro y controlado.

6.2 Requisitos no funcionales

Además, se consideraron requisitos no funcionales relevantes:

- **Escalabilidad**: capacidad de crecimiento del sistema sin rediseño total.
- **Mantenibilidad**: configuración clara y modular.
- **Seguridad**: protección de API keys y datos sensibles.
- **Flexibilidad**: soporte a varios proveedores y futuras integraciones.
- **Usabilidad**: experiencia conversacional sencilla para el usuario final.
- **Trazabilidad**: posibilidad de seguir el flujo desde la consulta hasta la creación del ticket.
- **Disponibilidad**: despliegue estable en la nube.

7. Arquitectura de la solución

7.1 Visión general

La arquitectura del sistema se apoya sobre varios bloques funcionales interrelacionados:

1. **Capa de presentación**: interfaz conversacional basada en LibreChat.
2. **Capa de aplicación**: backend encargado de gestionar conversaciones, llamadas a modelos, contexto y lógica de negocio.
3. **Capa de persistencia**: MongoDB para almacenar datos conversacionales, configuraciones y metadatos.
4. **Capa de IA**: conexión con modelos externos (Azure, Vertex AI y potencialmente otros).
5. **Capa de integración**: conectores MCP para consultar Guru y operar sobre Jira.
6. **Capa de clasificación**: lógica que analiza palabras clave y determina la vertical funcional.
7. **Capa de configuración y gobierno**: archivos YAML y parámetros de seguridad.

7.2 Frontend y experiencia conversacional

La elección de una solución tipo LibreChat permitió disponer desde el inicio de una interfaz conversacional madura, con funcionalidades como:

- Gestión de chats y sesiones.
- Entrada de texto del usuario.
- Posible adjunto de documentos.
- Visualización de respuestas generadas.
- Configuración de modelos disponibles.

Esto supuso una ventaja crítica en tiempo de desarrollo. En lugar de invertir esfuerzo en diseñar un frontend completo, el trabajo pudo enfocarse en las integraciones, el despliegue y la personalización del comportamiento del sistema.

7.3 Backend y orquestación

El backend de la plataforma actúa como orquestador central. Sus funciones principales son:

- Recibir la consulta del usuario.
- Recuperar el contexto de conversación o memoria aplicable.
- Determinar si debe consultar una fuente externa.
- Invocar el modelo de IA seleccionado.
- Integrar información procedente de Guru o de documentos subidos.

- Aplicar lógica de clasificación cuando la intención del usuario se relaciona con soporte o incidencia.
- Lanzar la creación del ticket en Jira si procede.

Este backend no es simplemente un intermediario técnico, sino el núcleo donde convergen decisiones funcionales, contexto, modelos y herramientas.

7.4 Persistencia con MongoDB

MongoDB se utiliza como sistema de almacenamiento principal por varias razones:

- Flexibilidad para manejar estructuras variables.
- Facilidad para guardar documentos JSON o equivalentes.
- Adaptación natural a datos conversacionales.
- Capacidad para almacenar metadatos, sesiones y configuración de manera ágil.

En el marco del proyecto, MongoDB puede contener:

- Conversaciones y mensajes.
- Configuraciones de sistema.
- Referencias a documentos procesados.
- Información de memoria o contexto persistente.
- Datos auxiliares requeridos por la plataforma.

7.5 Integración con proveedores de IA

La plataforma se ha configurado para conectarse con varios proveedores de modelos, principalmente Azure y Vertex AI. Esta capacidad multicloud/multiproveedor aporta ventajas significativas:

- Evita el bloqueo con un único proveedor.
- Permite seleccionar modelos distintos según la tarea.
- Facilita la evaluación comparativa entre modelos.
- Ofrece flexibilidad para optimizar costes y latencia.

Además, la posibilidad de integrar modelos open source o soluciones adicionales amplía el potencial del sistema y refuerza su carácter extensible.

7.6 Sistema RAG y gestión documental

Cuando el usuario sube un documento o cuando el sistema necesita consultar información relevante, entra en juego una lógica de recuperación documental. El contenido se procesa, se identifica su tipo y se almacena junto con la información necesaria para su consulta posterior. Esto permite enriquecer las respuestas con contexto específico, una característica especialmente importante en entornos empresariales donde el conocimiento operativo no se encuentra en el corpus general de un LLM.

7.7 Integración con Guru mediante MCP

Guru actúa como fuente de conocimiento corporativo estructurado o semiestructurado. Mediante MCP, la plataforma puede consultar esta base de conocimiento para obtener información útil durante la conversación. Esta integración es crítica porque:

- Reduce dependencia del conocimiento implícito del modelo.
- Mejora la fiabilidad de las respuestas.
- Permite conectar el asistente con documentación viva de la empresa.
- Aumenta el valor real del sistema en el día a día.

7.8 Integración con Jira mediante MCP

La integración con Jira permite que el asistente no se limite a informar, sino que también ejecute acciones concretas. Cuando el sistema detecta que una petición requiere la apertura de una incidencia, puede generar el ticket automáticamente, trasladando a Jira la información relevante obtenida del contexto conversacional.

Esto transforma el asistente en una herramienta operativa y no solo consultiva.

7.9 Clasificación por dominio mediante archivo Markdown

Uno de los elementos más originales y pragmáticos del proyecto es el mecanismo de clasificación por verticales. El sistema utiliza un documento Markdown que contiene palabras clave asociadas a distintos dominios funcionales. A partir del análisis del texto introducido por el usuario, la aplicación determina la vertical más probable y utiliza esa información para dirigir el ticket directamente al equipo adecuado.

Esta aproximación tiene varias ventajas:

- Es simple de implementar y mantener.
- No requiere entrenar un clasificador complejo.
- Puede ser ajustada fácilmente por personal funcional.
- Permite una mejora iterativa muy rápida.

Aunque no alcanza la sofisticación de un modelo supervisado entrenado específicamente, resulta muy adecuada como primera solución operativa y aporta gran valor en términos de rapidez de implementación.

8. Implementación del proyecto

8.1 Despliegue en Google Cloud

La plataforma se levantó en una instancia de Google Cloud, lo que permitió disponer de un entorno accesible, reproducible y apto para pruebas funcionales. El despliegue en nube aporta ventajas claras frente a un entorno local:

- disponibilidad remota,
- capacidad de configuración centralizada,
- posibilidad de escalar recursos,
- mejor aproximación a un escenario de uso real.

Desde el punto de vista de un TFM, esta decisión también demuestra una aproximación profesional a la puesta en producción de soluciones de IA.

8.2 Uso de LibreChat como base tecnológica

En lugar de construir toda la aplicación desde cero, se decidió utilizar LibreChat como base para el frontend y parte de la lógica de backend. Esta decisión se justifica por criterios de eficiencia y foco: el objetivo del proyecto no era reinventar una interfaz conversacional, sino construir valor diferencial sobre una plataforma existente.

LibreChat proporcionó una estructura inicial suficiente para:

- gestionar conversaciones,
- conectar modelos,
- almacenar interacciones,
- aceptar configuración externa.

A partir de ahí, se añadió la lógica empresarial específica.

8.3 Configuración mediante YAML

La plataforma se apoya en archivos de configuración, especialmente en YAML, para definir aspectos como:

- endpoints de modelos,
- proveedores habilitados,
- parámetros funcionales,
- límites de archivos,
- activación o desactivación de memoria,
- ajustes de integración.

Esta forma de configuración aporta transparencia, trazabilidad y facilidad de mantenimiento. También facilita la replicación del sistema en otros entornos, siempre que se gestionen adecuadamente los secretos y credenciales.

8.4 Gestión de memoria y prompts

La plataforma diferencia entre memoria persistente y prompts específicos. Esta distinción es importante desde el punto de vista funcional:

- La **memoria** permite incorporar contexto recurrente o preferencias que deben estar disponibles de forma transversal.
- Los **prompts** permiten modular tareas concretas, definir instrucciones especializadas o adaptar el comportamiento del sistema según el caso de uso.

Esta separación mejora la gobernanza del sistema y evita mezclar reglas permanentes con instrucciones transitorias.

8.5 Integración con Guru

La conexión con Guru mediante MCP se implementó como fuente de datos corporativa. El sistema consulta dicha fuente cuando necesita responder preguntas relacionadas con conocimiento interno o cuando requiere soporte documental fiable. El valor de esta integración radica en convertir el asistente en una puerta de acceso al conocimiento de la organización.

8.6 Integración con Jira

La conexión con Jira vía MCP se orientó a la creación de tickets. De este modo, el asistente puede estructurar la información relevante y registrarla en la herramienta de gestión de incidencias. Esto evita tareas manuales repetitivas y acelera el inicio del flujo de resolución.

8.7 Lógica de derivación a verticales

Para evitar que todas las incidencias pasen por un helpdesk genérico, se incorporó una lógica de clasificación basada en un fichero Markdown de palabras clave. El proceso general sería el siguiente:

1. El usuario describe su problema o necesidad.
2. El sistema analiza el texto de entrada.
3. Se comparan términos relevantes con las palabras clave asociadas a cada vertical.
4. Se asigna la incidencia al dominio más probable.
5. Se crea el ticket en Jira orientado directamente al área competente.

Este flujo reduce fricción y convierte al asistente en un elemento activo dentro del proceso de soporte.

9. Casos de uso principales

9.1 Consulta de conocimiento corporativo

Un usuario necesita resolver una duda sobre un procedimiento interno. En lugar de buscar manualmente en la documentación, formula la pregunta al asistente. El sistema consulta Guru, recupera la información pertinente y genera una respuesta contextualizada y comprensible.

9.2 Creación automática de un ticket

Un usuario informa de una incidencia funcional o técnica. El asistente analiza la petición, recopila el contexto necesario y crea automáticamente un ticket en Jira, evitando al usuario el acceso manual a la herramienta y reduciendo el tiempo de registro.

9.3 Derivación directa a una vertical

El usuario describe un problema relacionado con un área específica. Gracias al análisis de palabras clave, el sistema identifica si la incidencia pertenece, por ejemplo, a una vertical funcional concreta, y dirige el ticket directamente a dicha área sin pasar por helpdesk.

9.4 Consulta apoyada en documentos adjuntos

El usuario adjunta un documento y plantea preguntas sobre su contenido. El sistema procesa el archivo, lo almacena y lo utiliza como contexto mediante el mecanismo RAG, generando respuestas más precisas y adaptadas al documento.

10. Resultados y aportación del proyecto

Aunque el valor de una plataforma de este tipo suele apreciarse plenamente en producción y con uso continuado, incluso a nivel de prototipo avanzado o piloto se pueden identificar resultados relevantes.

10.1 Reducción del tiempo de desarrollo

Uno de los principales logros del proyecto es haber construido una solución funcional sin necesidad de desarrollar desde cero toda la interfaz conversacional ni el esqueleto del backend. El uso de una base como LibreChat permitió concentrar el esfuerzo en la lógica diferencial: integraciones, clasificación, despliegue y automatización.

10.2 Unificación de acceso a modelos y herramientas

La plataforma actúa como punto unificado de interacción con distintos modelos de IA y sistemas corporativos. Esta unificación simplifica la experiencia del usuario y evita que deba alternar entre múltiples herramientas para consultar información o generar acciones.

10.3 Mejora del acceso al conocimiento interno

La integración con Guru mejora la utilidad del asistente, ya que lo conecta con conocimiento empresarial real. Esto permite respuestas más aplicadas y disminuye la dependencia de búsquedas manuales o de la experiencia tácita de personas concretas.

10.4 Automatización de procesos de soporte

La creación de tickets en Jira desde el propio entorno conversacional representa una mejora operativa tangible. El sistema deja de ser meramente informativo y se convierte en un actor útil dentro del flujo de soporte.

10.5 Optimización del triage de incidencias

El mecanismo de clasificación por verticales introduce una capa de inteligencia de negocio sencilla, pero eficaz. Aunque se base en palabras clave y no en un modelo complejo de clasificación, su impacto puede ser elevado si reduce el número de incidencias mal dirigidas o la carga del helpdesk generalista.

10.6 Modularidad y capacidad de evolución

La arquitectura elegida deja abierta la incorporación futura de:

- nuevos proveedores de modelos,
- nuevas herramientas conectadas vía MCP,
- sistemas de clasificación más avanzados,
- métricas de uso y calidad,
- mecanismos de gobernanza y auditoría más sofisticados.

11. Limitaciones del sistema

Como todo proyecto aplicado, la solución presenta también limitaciones que conviene reconocer explícitamente.

11.1 Dependencia de la calidad de la configuración

Una parte importante del comportamiento del sistema depende de la correcta configuración de modelos, endpoints, credenciales y reglas. Una configuración pobre puede degradar la calidad de la experiencia.

11.2 Clasificación basada en palabras clave

Aunque útil y pragmática, la clasificación por palabras clave puede ser insuficiente en casos ambiguos, textos muy vagos o dominios con terminología solapada. A largo plazo, podría complementarse con técnicas de clasificación semántica más avanzadas.

11.3 Dependencia de servicios externos

El sistema depende de proveedores de IA y de herramientas corporativas externas. Cambios en APIs, disponibilidad o costes pueden afectar a la operatividad.

11.4 Riesgo de respuestas incorrectas

Como en todo sistema basado en LLM, existe el riesgo de generar respuestas inexactas si el contexto es insuficiente o la recuperación documental falla. Por ello, resulta fundamental establecer mecanismos de validación y gobernanza.

11.5 Seguridad y gestión de secretos

El proyecto requiere manejar API keys, configuraciones sensibles y accesos a sistemas empresariales. Un mal tratamiento de estos elementos puede suponer riesgos importantes.

12. Conclusiones

El presente Trabajo Fin de Máster ha permitido diseñar e implementar una plataforma conversacional corporativa basada en IA que combina varios elementos de alto valor práctico: interfaz de conversación, integración con múltiples modelos, persistencia de datos, recuperación de conocimiento, automatización de tickets y enrutado inteligente por dominio funcional.

Uno de los principales aprendizajes del proyecto es que, en el ámbito empresarial, el valor de la inteligencia artificial no reside únicamente en el modelo, sino en la **arquitectura de integración** que lo rodea. Un modelo conversacional sin acceso a fuentes internas ni capacidad de interacción con herramientas corporativas tiene un alcance limitado. En cambio, cuando se le dota de contexto documental, memoria, configuraciones adaptadas y capacidad de acción, se convierte en una verdadera pieza del ecosistema digital de la organización.

La decisión de utilizar una plataforma ya existente como LibreChat ha demostrado ser acertada desde el punto de vista de eficiencia y foco. Ha permitido reducir drásticamente el tiempo dedicado a aspectos genéricos y concentrar el esfuerzo en lo que realmente aporta valor diferencial: el despliegue cloud, la configuración multiproveedor, las integraciones vía MCP y la lógica de clasificación y derivación de incidencias.

Asimismo, la integración con Guru y Jira evidencia una orientación claramente aplicada. El asistente no solo informa, sino que **consulta conocimiento real y ejecuta acciones operativas**, lo que incrementa notablemente su utilidad en un contexto empresarial. La incorporación de un mecanismo de clasificación por verticales basado en Markdown, aunque sencillo, constituye una solución eficaz y mantenible para resolver un problema concreto de negocio: reducir el paso innecesario por un helpdesk intermedio.

En definitiva, el proyecto demuestra que es posible construir una solución de inteligencia artificial útil, modular y escalable sin necesidad de desarrollar todos los componentes desde cero. La combinación de tecnologías cloud, bases de datos flexibles, plataformas conversacionales reutilizables y protocolos de integración como MCP permite acelerar la adopción de la IA en entornos reales.

Como conclusión final, este trabajo no solo representa la implementación de un prototipo técnico, sino la validación de una estrategia: **usar IA de forma conectada, contextualizada y operativa para mejorar procesos reales de la organización**.

13. Bibliografía orientativa , fuentes y adjuntos

13.1 Bibliografía

- Gemini API | Google AI for Developers
- Vertex AI Platform | Google Cloud
- Hugging Face – The AI community building the future.
- https://github.com/danny-avila/LibreChat
- https://clickhouse.com/blog/clickhouse-acquires-librechat
- https://www.librechat.ai/docs/configuration/librechat_yaml/example
- Using Guru as an MCP Server
- Getting started with the Atlassian Rovo MCP Server | Atlassian Rovo MCP Server Cloud | Atlassian Support

13.2 Adjuntos

- **Archivo setup :** MiquelRaga_Setup.yaml
- **Prompt :** MiquelRaga_AgentPrompt.md
- **Keywords** : MiquelRaga_JiraKeywords.md
- **Video funcionamiento**: MiquelRaga_VideoFuncionamiento.webm